CHAPTER 1

DESIGNING A MOVIE TICKET BOOKING SYSTEM

1 Getting Ready: Movie Ticket Booking System 2
2 Requirements for the Movie Ticket Booking System . . . 4
3 Use Case Diagram for the Movie Ticket Booking System 6
4 Class Diagram for the Movie Ticket Booking System . . . 10
5 Sequence Diagram for the Movie Ticket Booking System 29
6 Activity Diagram for the Movie Ticket Booking System 33
7 Code for the Movie Ticket Booking System 36
8 Summary: Designing a Movie Ticket Booking System . . . 47
9 Key Insights and Tips: Designing a Movie Ticket Booking System 52

This chapter covers the design of a movie ticket booking system, including requirements, UML diagrams, and implementation code to build the system.

1 Getting Ready: Movie Ticket Booking System

1.1 Problem definition

The **movie ticket booking system** enables customers to search for movies, browse showtimes across various cinemas in multiple cities, select preferred seats, and securely book tickets online. The system manages comprehensive information about cinemas, locations, cinema halls, movie listings, and scheduled shows. Customers can view a live seating layout for each show, select and reserve available seats, receive booking confirmations, and complete payment.
In this LLD interview case study, you'll focus on:

- Modeling cinemas, cinema halls, movies, shows, and seating arrangements across cities.
- Designing secure seat reservation workflows that prevent double bookings and support concurrent access.
- Implementing robust payment flows.
- Ensuring timely notifications and confirmations for all successful bookings.

Note: This system model can be adapted for any online ticketing platform—such as concerts, sports, or events—with similar requirements for real-time seat selection and bookings.

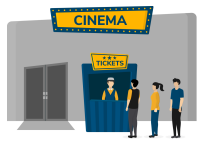


Figure 1.1: Customers are lining up to book tickets for a show

1.2 Expectations from the interviewee

A typical movie ticket booking system has numerous components, each with specific constraints and requirements. The following provides an overview of some of the main expectations that the interviewer will want to hear you discuss in more detail during the interview.

Seat selection

Selecting a seat is an essential part of the movie ticket booking system. The system must ensure that no two people can book the same seat. The interviewer expects you to ask questions to identify how the system will work in these situations:

- How will the system ensure multiple users do not book the same seat?
- Will there be a timeout session that temporarily reserves seats? Will the system use a first-come, first-served algorithm?
- Will there be transaction locks involved in the system?

Payment handling

One of the most significant attributes of the movie ticket booking system is the payment structure that it provides to its customers. This can vary, so the interviewer would expect you to ask the questions listed below:

- What payment methods can the customer use (for example, credit card or cash)?
- How is the payment performed? Does the customer pay themselves online or through a ticket agent at the location?

Price variance

We touched upon the payment methods of the movie ticket booking system. Now , the pricing model needs to be clarified by the interviewer, and therefore, you may ask questions like these:

- How will the booking price be calculated? Will it vary based on the popularity of the show?
- Does the seat type affect the pricing?
- Will there be discount coupon codes?

Duplication

There will be many duplicate instances in our system. The interviewer expects you to ask questions like these:

- How are we handling these instances, such as the same cinema having multiple cinema halls showing different movies simultaneously?
- Is the same movie shown in the same cinema/hall at different times?

1.3 Design approach

We will design this movie ticket booking system using the bottom-up design approach. For this purpose, we will follow the steps below:

- First, we'll identify core entities such as `Cinema`, `CinemaHall`, `Movie`, Show Seat, and `Booking`, and define their responsibilities.
- Next, we'll model how customers search for movies, view showtimes, and interact with seating layouts to reserve seats.
- We'll implement mechanisms to handle concurrency—preventing double bookings and managing seat reservations—while integrating reliable payment methods.

This design approach will address edge cases (such as simultaneous seat selection), scalability, and SOLID principles for maintainable, extendable architecture. In later lessons, we'll illustrate major workflows with diagrams and code examples.

1.4 Design pattern

Discussing the design patterns that a movie ticket booking system follows during an interview is always a good practice. Stating the design patterns gives the interviewer a positive impression and shows that the interviewee is well-versed in the advanced concepts of object-oriented design.
Try to answer the following question. If you are not familiar with design patterns, don't worry! You can learn about them by asking, "Define design patterns."

Question:

Which design pattern(s) should be used to design a movie ticket booking system? Please elaborate on your choice(s).

Let's explore the requirements of the movie ticket booking system in the next lesson.

2 Requirements for the Movie Ticket Booking System

This lesson outlines the functional and operational requirements for the movie ticket booking system. Identifying and understanding requirements is essential to defining the system's scope and ensuring a robust, user-friendly design.
We'll use a notational convention to identify each requirement with a unique label "Rn," where "R" is short for Requirement and "n" is a natural number.

2.1 Requirement collection

The following are the requirements that we have defined for the movie ticket booking problem:

- **R1:** The system supports multiple cinemas, each potentially located in different cities. Each cinema contains multiple halls.
- **R2:** A cinema can schedule multiple shows for each movie, but a hall can host only one show at a time.
- **R3:** The system must display all available showtimes for any selected movie, across all cinemas.
- **R4:** Users can search for movies by title, language, genre, or release date.

Column component type 'MxGraphWidget' not yet supported.

- **R5:** Users can book seats for any available showtime at any cinema hall.

Column component type 'MxGraphWidget' not yet supported.

- **R6:** Customers may make bookings online or via a ticket agent.
- **R7:** Online customers can pay by credit card only; in-person customers may pay by cash or credit card through a ticket agent.
- **R8:** Users can select multiple available seats for a show in a single booking transaction.

Column component type 'MxGraphWidget' not yet supported.

- **R9:** Seats are categorized into three fixed-cost types: Silver, Gold, and Platinum.
- **R10:** Each seat may have only one ticket allocated.
- **R11:** The system must prevent multiple users from booking the same seat for the same show.
- **R12:** Admin users must be able to add, delete, or update shows and movies.

Column component type 'MxGraphWidget' not yet supported.

- **R13:** The system visually distinguishes between available and booked seats.
- **R14:** The system must generate notifications for relevant events, such as:
  - When a new movie is released, notify users who opted in.
  - When a booking is made, notify the customers.
  - When a booking is canceled, notify the customers.
- **R15:** The system must send a notification to the customer whenever a booking is modified, informing them of the updated booking details.

We've identified our requirements for the problem, and in the next lesson, we will define different use cases of our movie booking system.

3 Use Case Diagram for the Movie Ticket Booking System

Let's build the use case diagram for the movie ticket booking system and understand the relationship between its main actors and functions. First , we'll define the different elements of our system, followed by the complete use case diagram.

3.1 System

Our system is the movie ticket booking system. It manages movie and show listings, ticket bookings, seat reservations, and notification delivery.

3.2 Actors

Now, we will define the main actors of our movie ticket booking system.

Primary actors

- **Customer:** Searches for movies, creates bookings, reserves seats, pays for tickets, views/modifies/cancels bookings.
- **Ticket agent:** The ticket agent will assist the customer and perform almost all the tasks a customer can do, such as creating a movie ticket booking on behalf of the customer, searching for movies, and reserving seats at the halls, except for modifying and canceling a booking.

Secondary actors

- **Admin:** Manages movies and showtimes by adding, deleting, or updating them.

3.3 Use cases

This section will define the use cases for movie ticket booking systems. We have listed the use cases according to their interactions with a particular actor.

> **Note:** Some use cases will occur multiple times because they are shared among different actors in the system.

Admin

- **Add show:** To add a new show for a particular movie in any cinema hall, specifying the movie, time, hall, and seat layout.
- **Modify show:** This function allows you to update the details of an existing show (such as date, time, or hall assignment) for any movie.

- **Delete (cancel) show:** To remove or cancel a scheduled show for a movie. This may also trigger notifications to affected customers.
- **Add movie:** To add a new movie to the system's listing, specify the title, language, genre, release date, and description.
- **Update movie:** To edit the details of an existing movie, such as title, language, genre, or duration.
- **Search movie:** To search for movies in the system using filters like title, language, genre, or release date.
- **Delete movie:** To permanently remove a movie from the system, which may also involve canceling all its scheduled shows and notifying customers.

Customer

- **Search movie:** You can search for movies by title, language, genre, or release date and view available shows across all cinemas.
- **Create booking:** To select a show, choose seats from the seating map, provide customer details, and create a booking.
- **View booking:** To view booking details, including movie name, showtime, cinema, hall, seat numbers, and booking status.
- **Modify booking:** To change selected seats or update booking details before final confirmation.
- **Cancel booking:** To cancel an existing booking, release reserved seats and, if applicable, request a refund.
- **Reserve seat:** To select one or more available seats from the real-time seating map during booking.
- **Pay for booking:** To complete payment for booked tickets using cash or a credit card (for online bookings).

Ticket agent

- **Search movie:** To search for movies by title, language, genre, or release date, on behalf of a customer at the cinema.
- **Create booking:** To select a show, choose seats, and create a booking for the customer in person.
- **View booking:** To retrieve and display the booking details for a customer at the cinema.

- **Reserve seat:** To select and reserve available seats for a show during in-person booking.
- **Accept payment:** To collect payment for bookings via cash or credit card on behalf of the customer.

Movie ticket booking system

- **Send new movie notification:** To automatically notify users (who have opted in) when a new movie is added to the system.
- **Send booking notification:** To send confirmation notifications to customers upon successful booking and payment.
- **Send cancellation notification:** Customers should receive notifications if their booking, movie, or show is canceled.
- **Send booking modification notification:** To automatically notify customers when their booking details (such as seat selection, showtime, or cinema hall) are changed.

3.4 Relationships

This section describes the relationships between actors and use cases in the movie ticket booking system. Understanding these relationships clarifies how different system components interact and helps ensure a robust, consistent design.

Generalization

Generalization shows inheritance or specialization between actors or use cases. The Search movie use case generalizes more specific search types:

- "Search movie by title"
- "Search movie by language"
- "Search movie by genre"
- "Search movie by release date"

Each specialized search is a specific way to perform the generic Search movie action. By showing generalization, we communicate that users can choose any of these criteria without changing the main workflow, and future new criteria can be added without restructuring the diagram.

Associations

The table below shows the association relationship between actors and their use cases.

Admin	**Customer**	**Ticket Agent**	**Movie Ticket Booking System**
Add show	Search movie	Search movie	Send new movie notification
Modify show	Create booking	Create booking	Send booking notification
Delete (cancel) show	View booking	View booking	Send cancellation notification
Add movie	Modify booking	Reserve seat	Send notifications for booking
Update movie	Cancel booking	Accept payment	
Search movie	Reserve seat		
Delete movie	Pay for booking		
	Receive notifications for bookings		
	Receive notifications for cancellations		

Include

The "include" relationship indicates that a use case always incorporates the behavior of another use case into its process.

- **Create booking** → **Reserve seat**:
  - Reserving seats is essential in the booking process, so "Create booking" always includes "Reserve seat."
- **Pay for booking** → **Send booking notification**:
  - Sending a booking notification is always included after successful payment.
- **Cancel booking** → **Refund payment**:
  - If payment was made, initiating a refund is always included as part of the cancellation process.
- **Add movie** → **Send new movie notification**:
  - Notification is always sent when a new movie is added.
- **Cancel booking**, **Delete movie**, **Delete show** → **Send cancellation notification**:

  - Canceling a booking, movie, or show always results in a cancellation notification.

- **Modify booking** and **Modify show** → **Send booking modification notification**:
  - Notification is always sent to customers when their booking details (such as seat selection, showtime, or cinema hall) are changed or the admin changes anything regarding the show.

3.5 Use case diagram

Here is the use case diagram of the movie ticket management system:

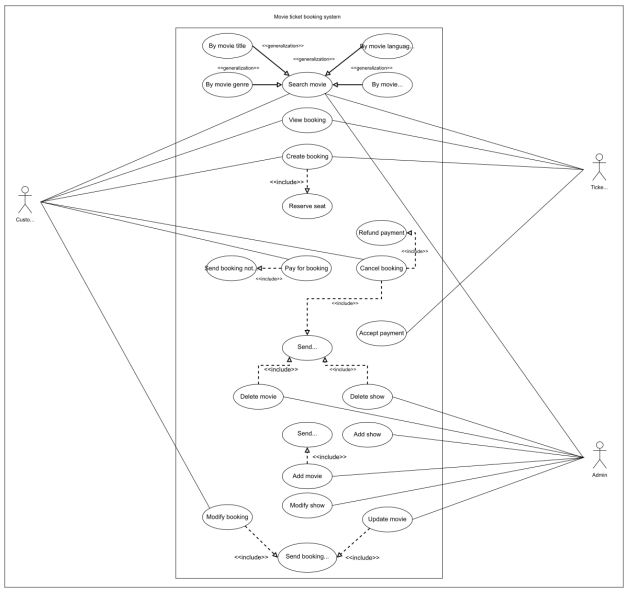


Figure 1.2: Use case diagram of a movie ticket booking system

In the next lesson, we will discuss the class diagram with a detailed explanation of all classes and their relationship with each other.

4 Class Diagram for the Movie Ticket Booking System

This lesson will identify and design the movie ticket booking system's key classes, abstract classes, and interfaces. Our approach follows the bottom-up design methodology, beginning with core entities and building up to system-wide relationships, as defined in the requirements lesson.

4.1 Components of a movie ticket booking system

We use a bottom-up approach, starting from core entities and relationships, ensuring the design aligns with functional and operational requirements.

Seat

The `Seat` class represents a physical seat in a cinema hall. It is an abstract class extended by three concrete seat types: `Silver`, `Gold`, and `Platinum`.

- Each seat type has a fixed cost (rate).
- The base `Seat` class defines the rate attribute and methods, and may be overridden in subclasses if necessary.
- Each seat also has a seat number and a seat status (available, booked, or reserved).

The visual representation of these classes is as follows:

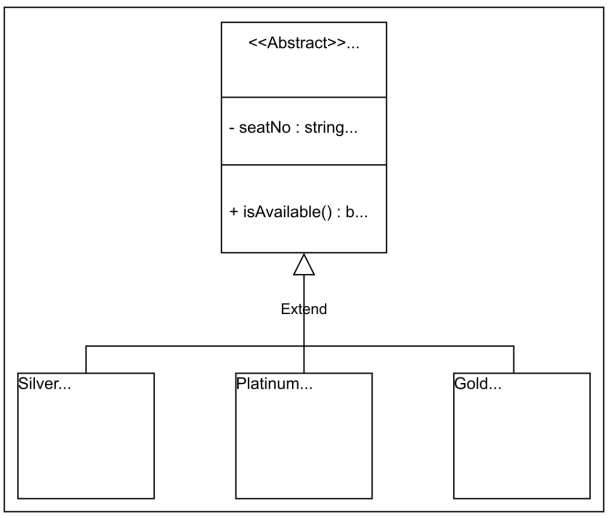


Figure 1.3: The class diagram of Seat and its derived classes

R9: Each seat type has a fixed cost. There are three types of seats: * Silver * Gold * Platinum
R13: The system should be able to differentiate between available and booked seats.

Show time

The `ShowTime` class represents a particular show of a movie.

- It contains the start time, date, and a reference to the movie being shown.
- ShowTime also maintains a mapping for seat statuses for that particular show.

Here is what the class definition looks like:
R5: Users can make a booking at any cinema at the available showtime.

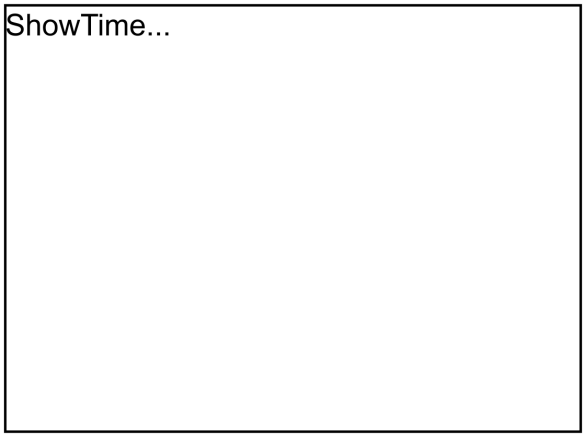


Figure 1.4: The class diagram of the ShowTime class

Hall

The `Hall` class represents a cinema hall.

- Each hall has a unique ID, a list of all seats (physical seats in the hall).
- Hall manages which show is playing and which seats are available for each show.

The UML representation of the class is shown below:

Figure 1.5: The class diagram of the Hall class

R2: Each movie in the cinema can have multiple shows, however; one hall will only show one show at a time.

Cinema

The `Cinema` class consists of the number of halls present in the cinema, along with the city in which it is located in, and an ID attribute to identify the cinema in that particular city. The class representation of the `Cinema` class is as follows:

R1: There are multiple cinemas in the city, each with multiple halls.

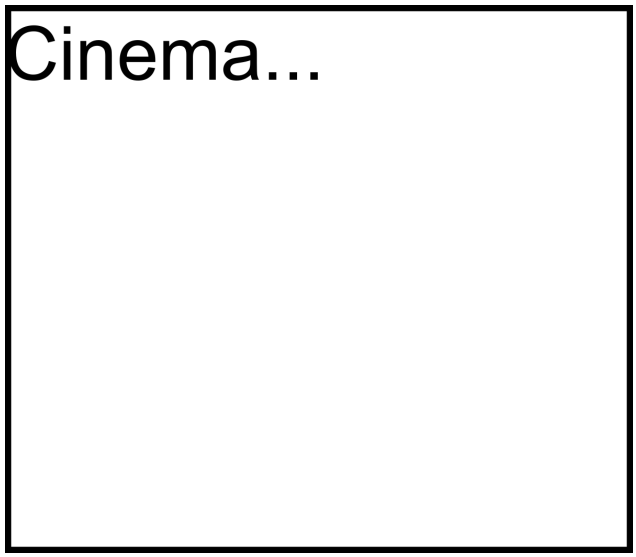


Figure 1.6: The class diagram of the Cinema class

City

The `City` class includes the city's name, state, and zip code, as well as a list of all its cinemas. The UML representation of the class is shown below:

Figure 1.7: The class diagram of the City class

R1: There are multiple cinemas in the city, each with multiple halls.

Movie

The `Movie` class contains details about a particular movie, such as its title, genre, language, and release date. It is also composed of a list of running shows.
Here is what the class definition looks like:

R4: Users can search movies based on the following four criteria:

- Title

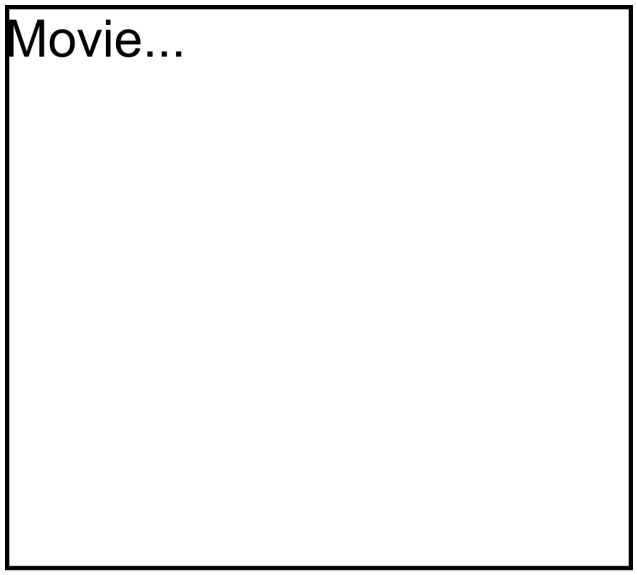


Figure 1.8: The class diagram of the Movie class

- Language
- Genre
- Release date

Movie ticket

The `MovieTicket` class refers to a customer's ticket for a particular movie, which has a ticketID as its unique identifier. The ticket describes the details of the seat in the hall, the movie for which the ticket was bought, and the show details.
The class representation of the `MovieTicket` class is as follows:

R10: There can only be one ticket allocated per seat.

Payment

The `Payment` class will be abstract and have two child classes: CreditCard and Cash, as these are the two payment methods of the movie ticket booking system.
The visual representation of these classes is as follows:

R7: Online customers can only pay using a credit card, while walk-in customers can pay using cash or a credit card through the ticket agent.

Person

The `Person` class contains details like name, address, phone number, and email, and is derived into the following three different classes:

- `Customer`

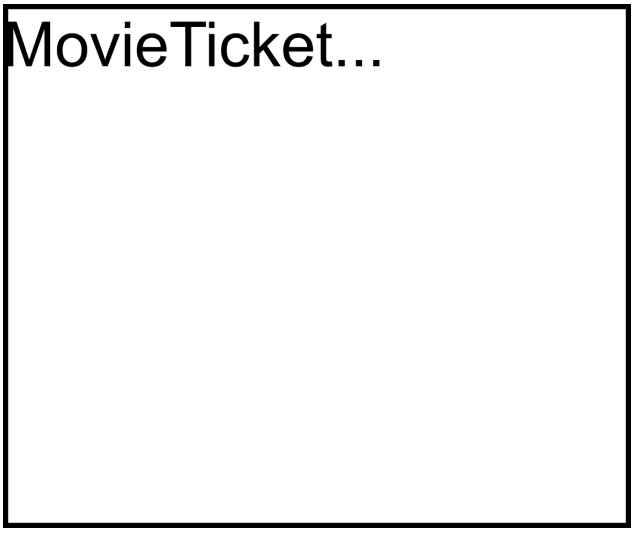


Figure 1.9: The class diagram of the MovieTicket class

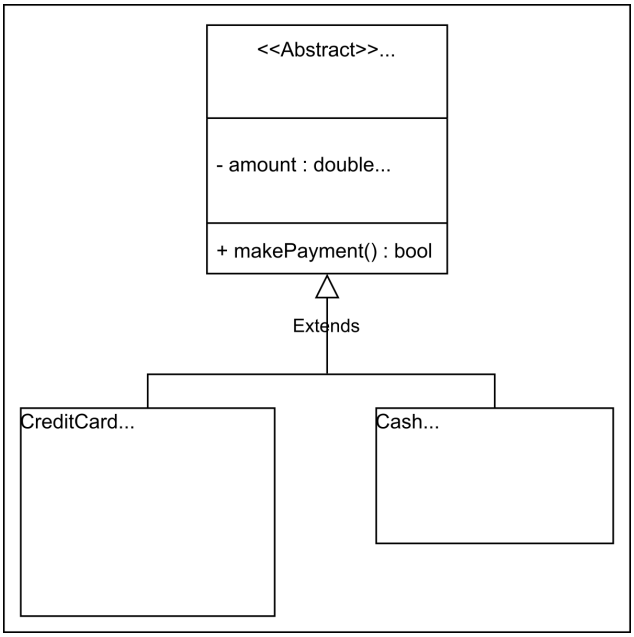


Figure 1.10: The class diagram of the Payment class and its child classes

- `Admin`
- `TicketAgent`

The following goes into more depth in each of the derived classes mentioned above:

Customer The `Customer` class refers to a user trying to create an online booking for any movie. A customer can also update the booking details and cancel the booking.

Admin The `Admin` class performs actions like adding, updating, and removing movies and shows.

Ticket agent The `TicketAgent` class is responsible for creating bookings on behalf of walk-in customers. However, unlike the `Customer` class, a ticket agent cannot update or cancel a booking.
The class diagram of all these is provided below:

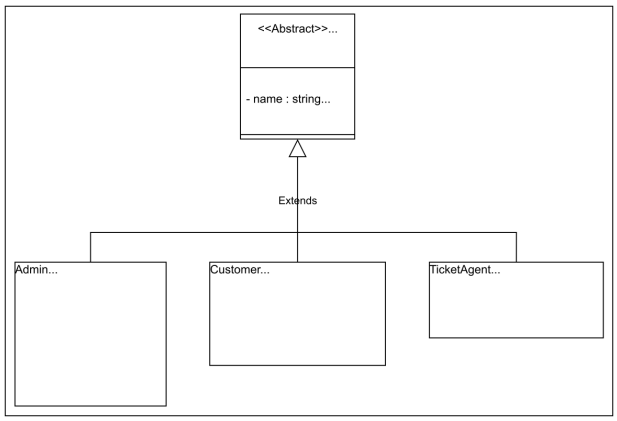


Figure 1.11: The class diagram of different people who interact with our system

R5: Users can book at any cinema hall at showtime.
R6: Customers can book online or by walking into the ticket agent.
R12: The admin can perform the following five actions on the showtimes and the movie:

- Add a Show
- Delete a Show
- Update a Show
- Add a Movie
- Delete a Move

Notification

`Notification` will be an abstract class since it can send a notification via email or phone (SMS). It is mainly responsible for sending notifications whenever any of the following conditions are met:

- A customer makes a new booking.
- The admin deletes a particular show, or a customer cancels their booking.
- The admin adds a new movie to the database.

The UML representation of the class is shown below:

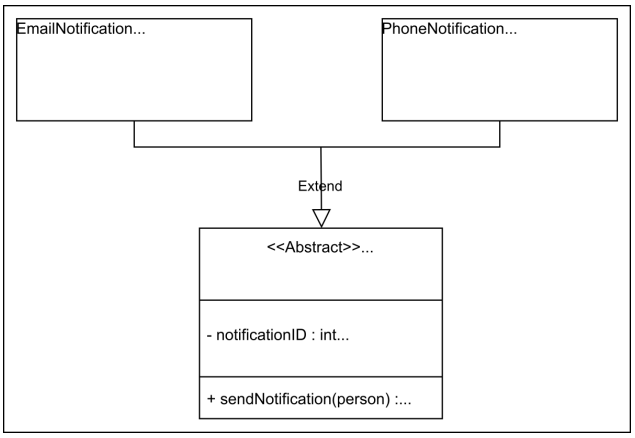


Figure 1.12: The class diagram of Notification and its child classes

R14: The system should generate a notification for the following three cases:

- A new movie has been released.
- A booking has been made.
- A booking has been canceled.

Catalog

The `Catalog` is the class where the search function is implemented. Each catalog contains a list of movies sorted according to one of the given search techniques, e.g., based on the movie's title, language, genre, or release date.
The class representation of the `Catalog` class is as follows:

R4: Users can search movies based on the following four criteria:

- Title
- Language
- Genre
- Release date

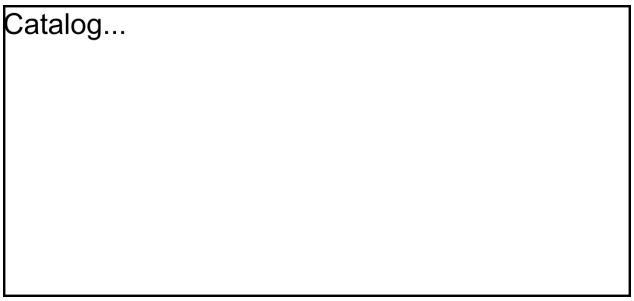


Figure 1.13: The class diagram of the Catalog class

Search

The `Search` class will be an interface that allows customers to search for any particular movie and return the list of movies upon searching, by any of the following methods:

- Search movies by their title.
- Search movies by their language.
- Search movies by their genre.
- Search movies by their release date.

The UML representation of the class is shown below:

Figure 1.14: The class diagram of the Search interface

R4: Users can search movies based on the following four criteria:

- Title
- Language
- Genre
- Release date

Booking

The `Booking` class is the main class of our movie ticket booking system and contains a reference to the number of seats and tickets that a customer booked. In addition, it includes information about the payment status, the booking status, the show time, and the total amount that was charged for the booking.
The UML representation of the `Booking` class is as follows:

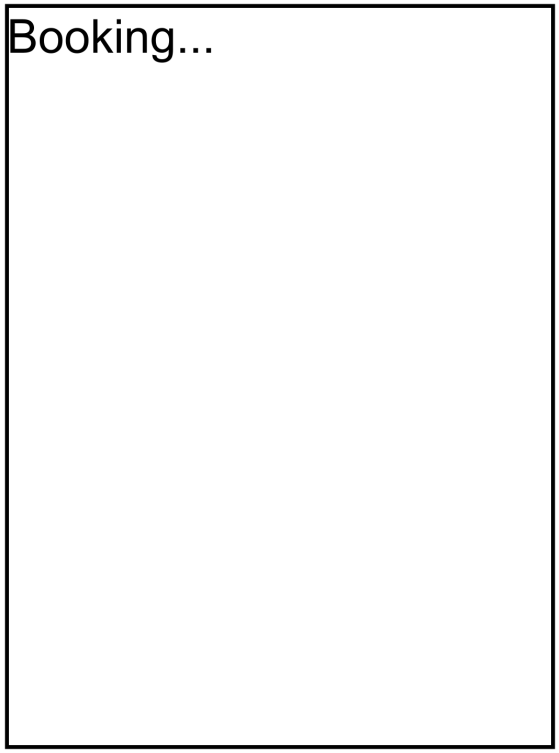


Figure 1.15: The class diagram of the Booking class

R5: Users can make a booking at any cinema at the available showtime.

Enumerations

The following is the list of enumerations required in the movie ticket booking system:

- `PaymentStatus:` The payment status checks if the customer's payment falls in any of the following stages: confirmed, declined, pending, or refunded.
- `BookingStatus:` The booking status describes the status of a particular movie booking of a customer, whether it is pending, confirmed, canceled, declined, or refunded.
- `SeatStatus:` The seat status indicates whether a particular seat in the hall is currently available, booked, or already reserved.

These enumerations can be represented using the following class diagram:

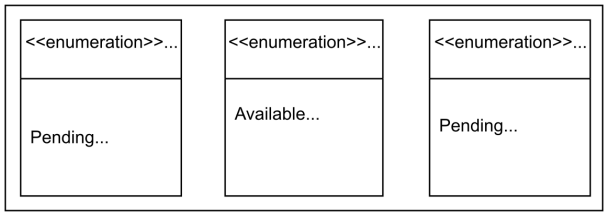


Figure 1.16: Enums in the movie ticket booking system

4.2 Relationship between the classes

Now, we will discuss the relationships between the classes we defined above in our movie ticket booking system.

Association

The class diagram has the following association relationships:

- The `City` class has a *one-way association* with `Cinema`.
  - A city maintains a list of cinemas because a real-world city contains multiple movie theaters. This association allows users to search and filter cinemas by city.
- The `Admin` class has a one-way association with `ShowTime`, `Movie`, and `Notification`.
  - The admin manages movies and shows in the system—adding, updating, or deleting them—and triggers notifications when such changes affect users (for example, when a show is canceled or a new movie is added).
- Both `Customer` and `TicketAgent` have a *two-way association* with `Notification`.
  - Customers and ticket agents receive notifications (such as booking confirmations or cancellations) and may trigger them through actions (e.g., booking or canceling a ticket).
- The `Booking` class has a one-way association with `MovieTicket`.
  - A booking generates one or more movie tickets, each representing a reserved seat for a specific show. This association allows one to retrieve all tickets related to a booking and track the seats and shows for which they were issued.
- The `Booking` class has a two-way association with `Payment`.
  - Each booking is linked to a payment to record the financial transaction for the booking. This association allows the system to track the payment status and method used for each booking.
- The `ShowTime` class has a one-way association with `Seat` and `MovieTicket`.
- The `Movie` class has a one-way association with `ShowTime` and `MovieTicket`.

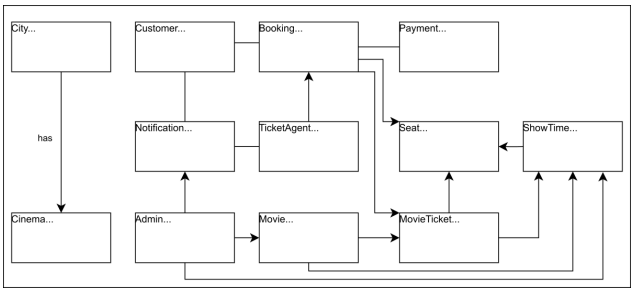


Figure 1.17: The association relationship between classes

Composition

The class diagram has the following composition relationships:

- The `Cinema` class is composed of the `Hall` class, which is composed of the `Seat` class.

Aggregation

The following classes show an aggregation relationship:

- The `Catalog` class contains the `Movie` class.

Generalization

The following classes show a generalization relationship:

- The `Catalog` class implements the `Search` class.

Inheritance

The following classes show an inheritance relationship:

- The `Gold`, `Platinum`, and `Silver` classes extend the `Seat` class.
- The `Admin`, `Customer`, and `TicketAgent` classes extend the `Person` class.
- Both `Cash` and `CreditCard` extend the `Payment` class.
- Both `EmailNotification` and `PhoneNotification` extend the `Notification` class.

  Note: We have already discussed the inheritance relationship between classes in the component section above one by one.

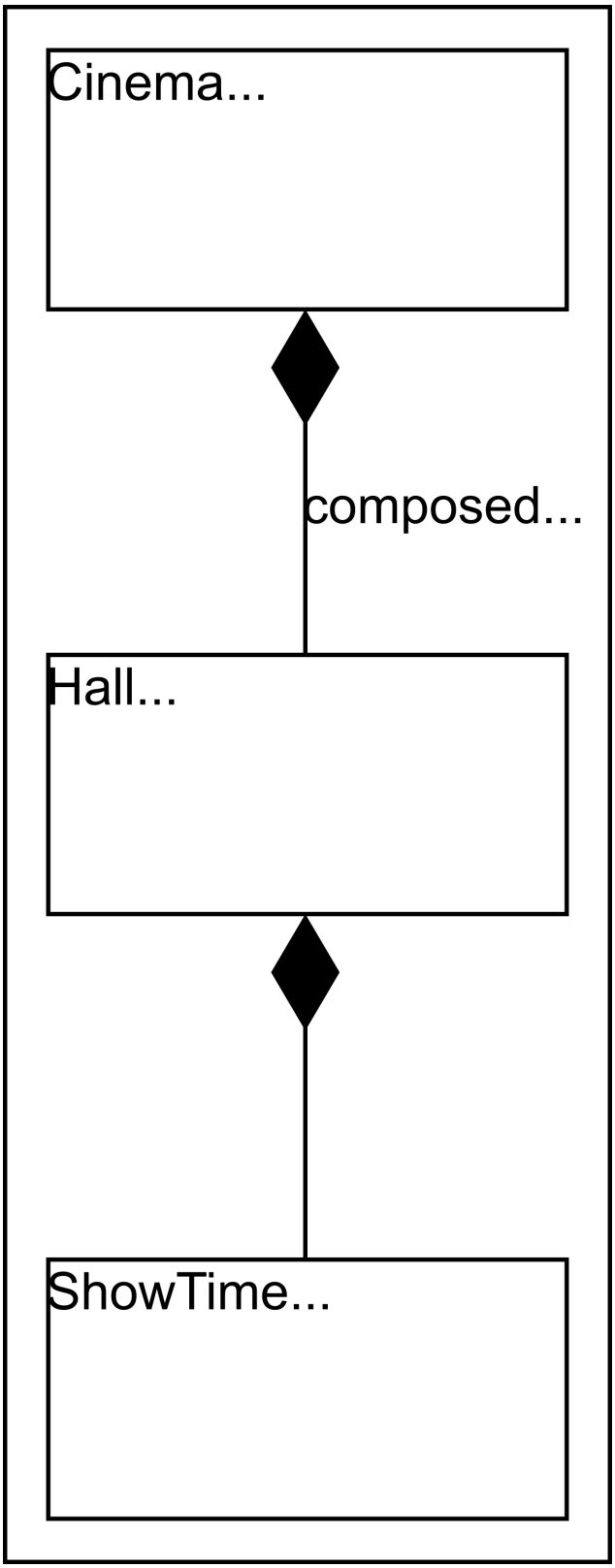


Figure 1.18: The composition relationship between classes

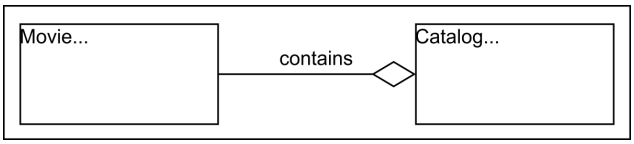


Figure 1.19: The aggregation relationship between classes

Figure 1.20: The generalization relationship between classes

4.3 Class diagram of the movie ticket booking system

This section outlines the multiplicity (cardinality) relationships between the main classes in our Movie Ticket Booking system. We explain the allowed number of instances on each side for each relationship and the real-world or design rationale behind the connection. Understanding these relationships is key to modeling how different entities interact and collaborate to support key workflows in the system.

Source	**Target**	**Multiplicity**	**Reason**
City	Cinema	1--0..*	A city can have multiple cinemas; each cinema belongs to one city.
Cinema	Hall	1--1..*	A cinema consists of one or more halls; each hall belongs to one cinema.
Hall	Seat	1--0..*	Each hall contains multiple seats; each seat belongs to one hall.
Movie	ShowTime	1--1	A movie can have multiple showtimes; each showtime shows one movie.
ShowTime	MovieTicket	1--0..*	Each showtime can generate multiple tickets; each ticket is for one showtime.

ShowTime	Seat	1--1..*	Each showtime manages status for multiple seats (by reference to hall's seats)
Booking	MovieTicket	1--1..*	Each booking includes one or more tickets (for each reserved seat).
Booking	Payment	1--1	Each booking is associated with exactly one payment.
Booking	Customer	0..*--1	A booking is made by one customer; each customer can have multiple bookings.
Admin	ShowTime	0..*--0..*	Admin manages (adds/updates/deletes) showtimes.
Admin	Movie	0..*--0..*	Admin manages (adds/updates/deletes) movies.
Admin	Notification	0..*--0..*	Admin can trigger notifications (e.g., for new movie/show, cancellation).
Customer	Notification	0..*--0..*	Customers receive notifications about bookings, cancellations, etc.
TicketAgent	Notification	0..*--0..*	Ticket agents can receive notifications as well.
Catalog	Movie	1--0..*	The catalog contains multiple movies for search and listing.
Catalog	Search (interface)	1--1	Catalog implements the Search interface to provide movie search features.
MovieTicket	Seat	1--1	Each ticket is for one specific seat.

MovieTicket	Movie	1--1	Each ticket is for one specific movie.

Here's the complete class diagram for our movie ticket booking system:

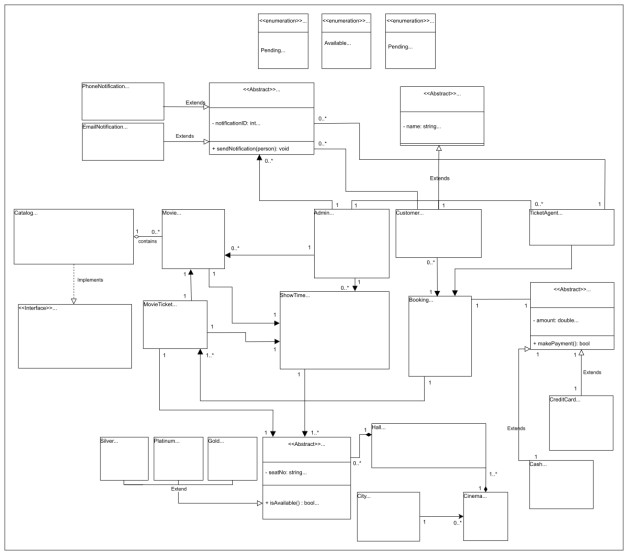


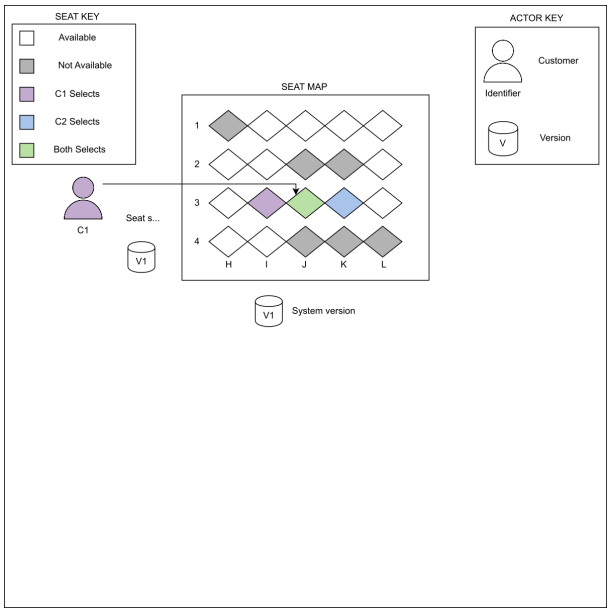
Figure 1.21: The class diagram of the movie ticket booking system

4.4 Handle concurrency

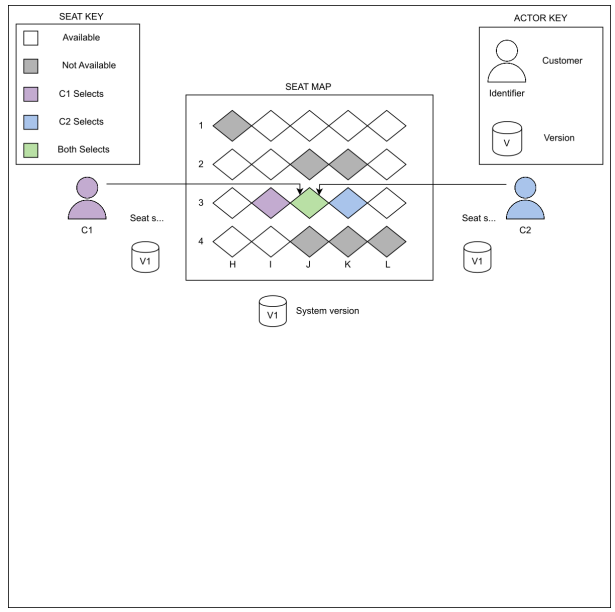
One of the major requirements of the movie ticket booking system is that no two customers can book the same seat in the same movie show. This prevents double booking, where two people have the same seats allocated to them.
To handle concurrent seat allocations, we can use locks, specifically optimistic locks or versioning.
The entire process has been summarized in the illustration given below:

- Customer 1 tries to select some seats on the seat selection map and gets assigned a version number V1.
- Simultaneously, customer 2 also tries to select some seats on the seat selection map and gets assigned a version number V1. However, both customers select the same seat.
- A lock time is assigned to track the timestamps related to customer entries in the payment section.
- The mutex lock is acquired, and the expiration period starts. As customer 1 took less time, the payment process started while customer 2 needed to wait.

- Customer 1 completes the payment. The mutex lock also gets released.
- Both customer 1 and the system versions get updated. As the lock has been released, customer 2 can now book the tickets again.
- However, as the system version has been updated, customer 2 would need to perform the entire task from the beginning.
- The previous seat has become unavailable, so customer 2 selects a new seat.



(a) Handling concurrent seat allocations

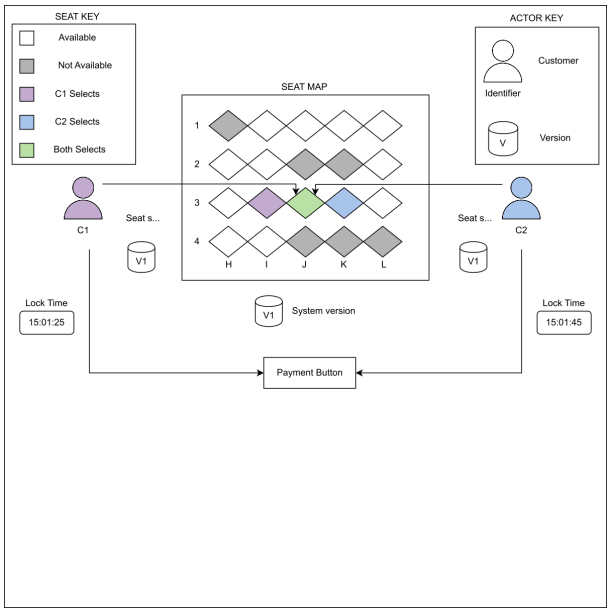


(b) Handling concurrent seat allocations

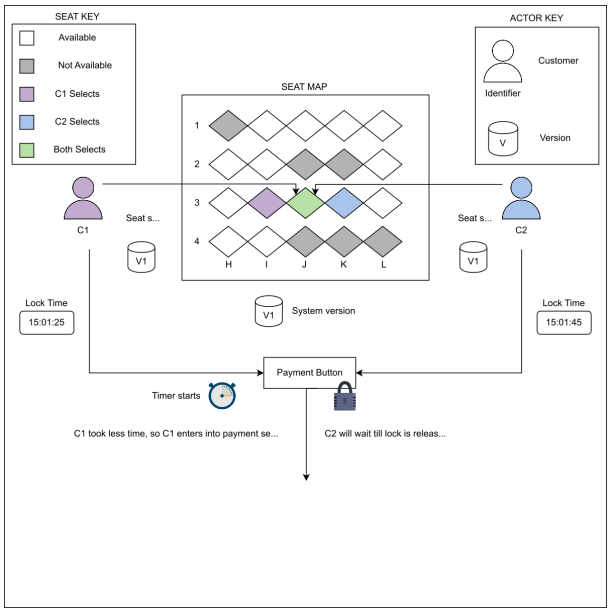
4.5 Design pattern

In the movie ticket booking system, there can be multiple seat types for a cinema hall, and each seat type will have its own formula to calculate the ticket fare. To ensure our movie ticket booking system remains flexible, maintainable, and easy to extend, several key object-oriented design patterns are incorporated into the architecture:

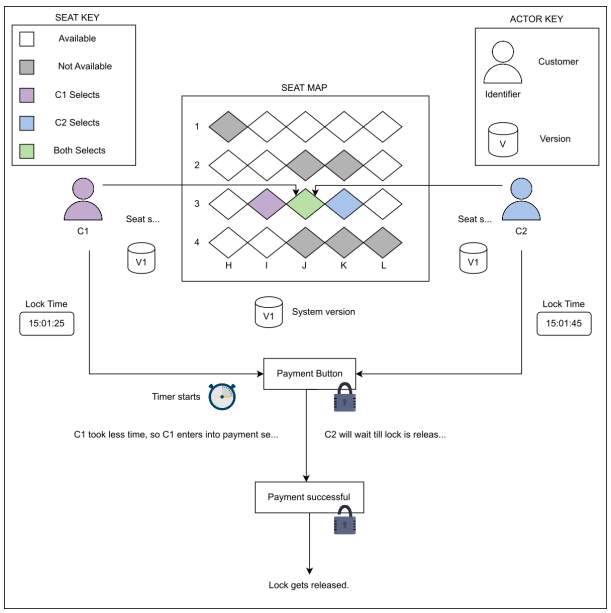
1. For handling dynamic ticket pricing across various seat types and situations, the Strategy pattern provides the needed flexibility. Each seat type is paired with a pricing strategy, making it simple to introduce new pricing rules or adjust for factors like showtime or movie popularity without altering core seat classes.
2. The system relies on the Factory pattern to create instances of seats, payments, or notifications. This approach centralizes and abstracts the instantiation logic, making it straightforward to add new seat types or payment methods with minimal code changes elsewhere.



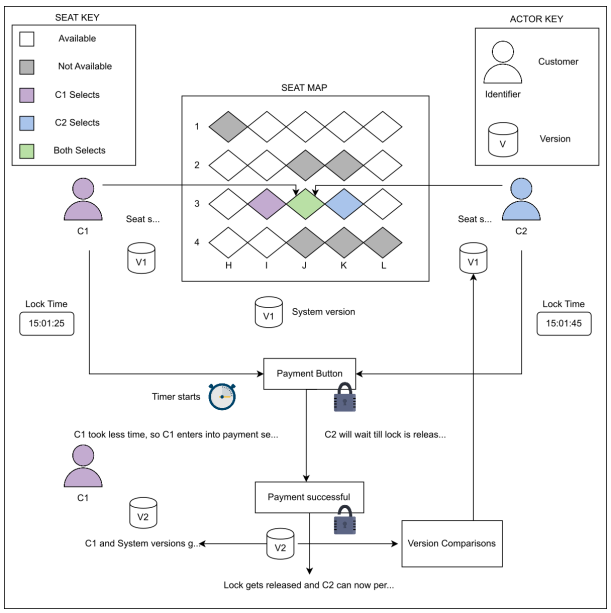
(a) Handling concurrent seat allocations



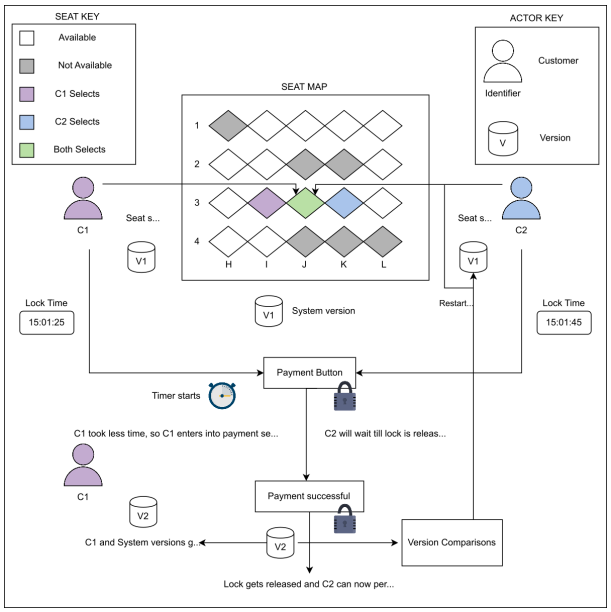
(b) Handling concurrent seat allocations



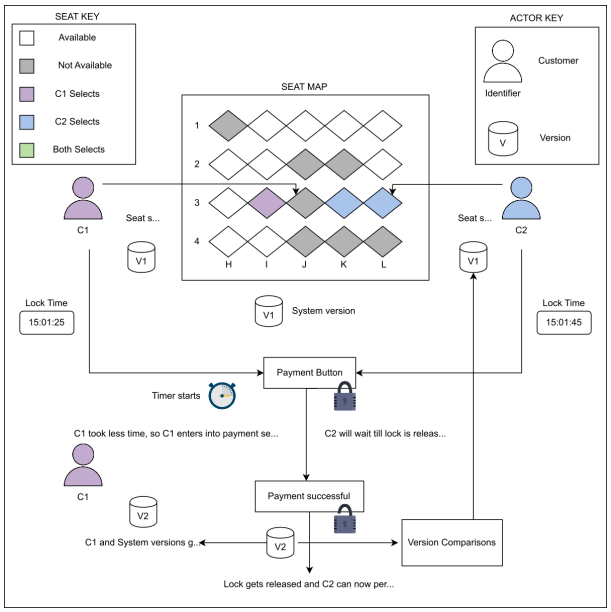
(a) Handling concurrent seat allocations



(b) Handling concurrent seat allocations



(a) Handling concurrent seat allocations



(b) Handling concurrent seat allocations

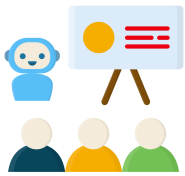
3. The notification module implements the Observer pattern to decouple business events from user communications. Events such as booking confirmations, cancellations, or new movie releases automatically trigger notifications to all relevant users or channels, keeping the system responsive and loosely coupled.

4. The Singleton pattern streamlines managing shared system-wide resources, such as the movie catalog or the notification service. This ensures a single, consistent point of access for these global components throughout the application.

5. Finally, the Builder pattern makes constructing complex objects like bookings or tickets easier—especially when multiple options or customizations are involved. This pattern results in cleaner, more readable code and flexible object assembly.

4.6 AI-powered trainer

At this stage, everything should be clear. If you encounter any confusion or ambiguity, feel free to utilize the interactive AI-enabled widget below to seek clarification. This tool is designed to help you strengthen your understanding of the concepts.



Component type 'PromptAI' not yet supported.

4.7 Additional requirements

The interviewer can introduce some additional requirements in the movie ticket booking system, or they can ask some follow-up questions. Let 's see some examples of additional requirements:

Discount: Customers can use a coupon to add a discount to their payment. The class diagram provided below shows the relationship of `Discount` with the `Payment` class:

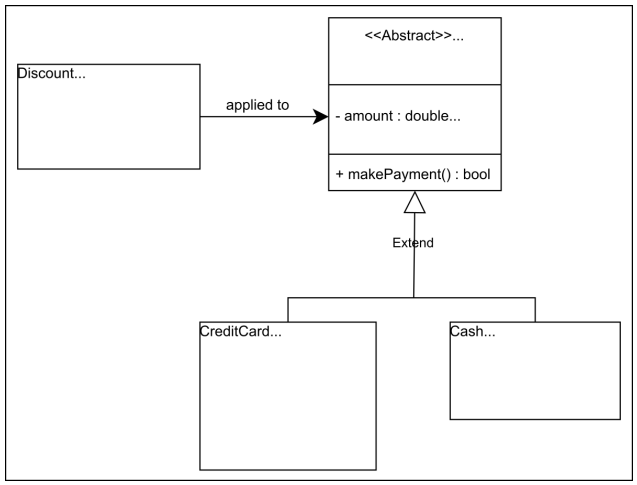


Figure 1.26: The relationship between the Discount class and the Payment class

Customer and guest users: There should be two types of users in our system. The guest user can search for movies and choose to register. The customer should be an authenticated user and be able to search movies and book shows. The class diagram below shows how we can implement separate functionality for different users:

We have completed the class diagram of the movie ticket booking system according to the requirements. In the next lesson, let's design the sequence diagram of the movie ticket booking system.

5 Sequence Diagram for the Movie Ticket Booking System

A sequence diagram is a great way to understand the interactions between different entities and objects in the system. There can be different sequence diagrams that we can create for our movie ticket booking system. In this lesson, we will create sequence diagrams for the following three interactions:

- **Create a booking:** The customer creates a booking for a show.
- **Payment for the booking:** The customer pays for the booking.
- **Sequence challenge:** The customer cancels their booking.

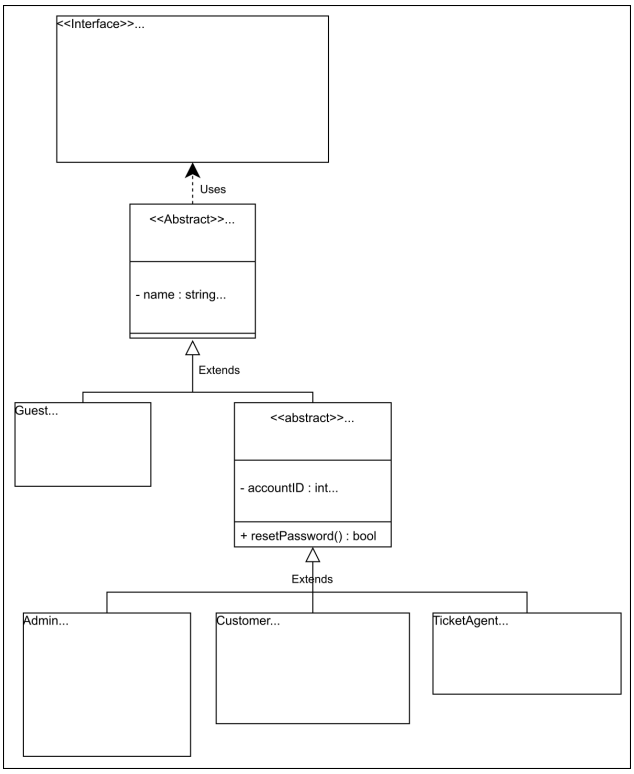


Figure 1.27: The class diagram of different users of our system

5.1 Create a booking

The sequence diagram for creating a booking should have the following actors and objects that will interact with each other:

- **Actor:** `Customer`
- **Objects:** `Catalog`, `ShowTime`, and `Booking`

Here are the steps of the interaction to create a booking:

1. The customer searches for a movie from the catalog.
2. The catalog returns the required movie(s).
3. The customer requests showtimes for the selected movie from the catalog.
4. The available showtimes for the movie are returned.
5. The customer requests available seats for the selected showtime.
6. If seats are available:
   (a) The customer receives a list of available seats.
   (b) The customer selects their desired seats and requests to create a booking.

(c) The booking is created for the customer, selected seats, and showtime. The booking status is set to "PENDING."

(d) The customer is notified that the booking is created and that payment is required.

7. Else if no seats are available:

   (a) The customer is informed that no seats are available.

Based on the order above, the sequence diagram for creating a booking in the movie ticket booking system is given below.

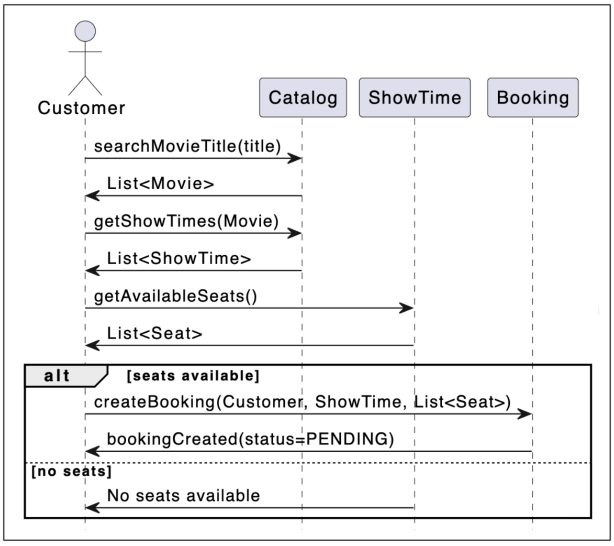


Figure 1.28: The sequence diagram to create a booking

5.2 Payment of a booking

The sequence diagram for payment of a booking should have the following actors and objects that will interact with each other:

- **Actor:** `Customer`
- **Objects:** `Booking`, `Payment`, and `Notification`

Here are the steps in the payment of a booking interaction:

1. The customer initiates a payment for the booking fee through the booking system.
2. The booking system creates a payment object and processes the payment.
3. If the payment is confirmed:

   (a) The payment informs the system about the successful payment.

(b) The booking status is updated to "CONFIRMED."

(c) The system sends a notification to the customer containing the booking confirmation.

4. Else if the payment is declined:

   (a) The payment informs the system about the declined payment.

   (b) The booking status is updated to "DECLINED."

   (c) The system sends a notification to the customer that the booking has been declined.

Based on the order above, the sequence diagram for payment of a booking in the movie ticket booking system is given below.

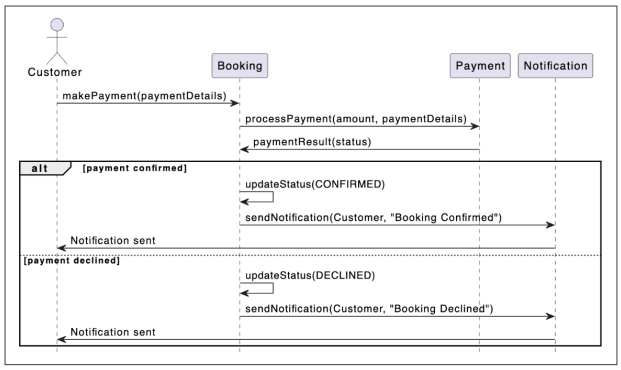


Figure 1.29: The sequence diagram for the payment of a booking interaction

5.3 Sequence challenge: Cancel booking

You will complete a sequence diagram for a booking canceled by the customer. A skeleton of the cancel sequence diagram is given below:

Notice that the arrows in the diagram above are numbered from 1 to 7. The message boxes shown below are the messages to be exchanged between the actor(s) and object(s). Can you rearrange the messages below in the correct sequence in order they should appear in the skeleton of the sequence diagram given above?

Note: If you're stuck, click the "Show Solution" button.

Alternatively, you can also click the "Show complete diagram" button below to see the complete sequence diagram for the cancel booking interaction.

Next, let's examine the activity diagrams for the movie ticket booking system to understand its control flow.

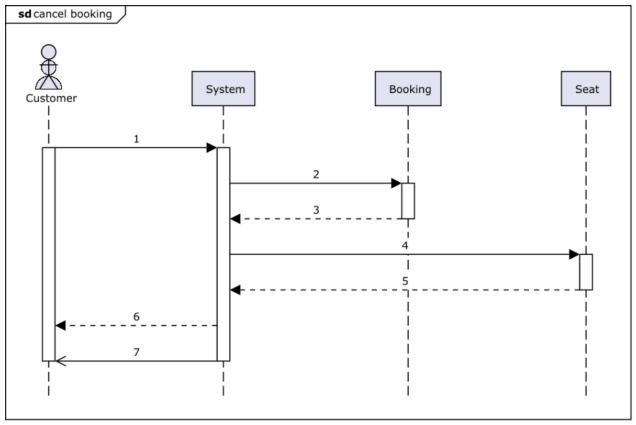


Figure 1.30: The sequence diagram for canceling the booking

6 Activity Diagram for the Movie Ticket Booking System

An activity diagram is a great way to visualize the flow of messages from one activity to another in the system. There can be different activity diagrams that we can create for our movie ticket booking system. In this lesson, we will create activity diagrams for the following two activities:

- The customer makes a booking for the movie.
- **Activity challenge:** The admin cancels a show.

6.1 The customer makes a booking for the movie

The following are the states and actions that will be involved in this activity diagram.

States

- **Initial state:** The customer opens a search for a movie.
- **Final state:** The customer receives a ticket for the movie.

Actions

The customer searches for a movie using specific criteria and selects it. The customer then selects their required seat and pays according to the seat type. The payment is made either through cash or a credit card. After successful payment, the customer receives the movie ticket.
Based on the order above, the activity diagram of a customer making a booking for the movie is given below.

> **Note:** Here, we assume the customer is purchasing a single cinema seat.

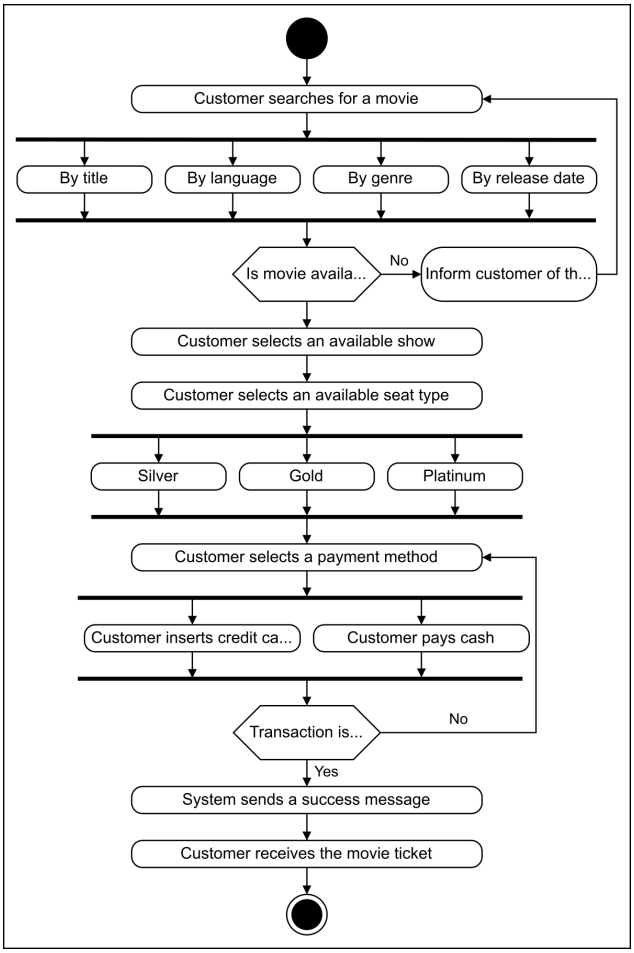


Figure 1.31: The activity diagram for the customer making a booking

6.2 Activity challenge: Admin cancels a show

You will create an activity diagram of an admin canceling a show for a movie.
A skeleton of the activity diagram is given below.

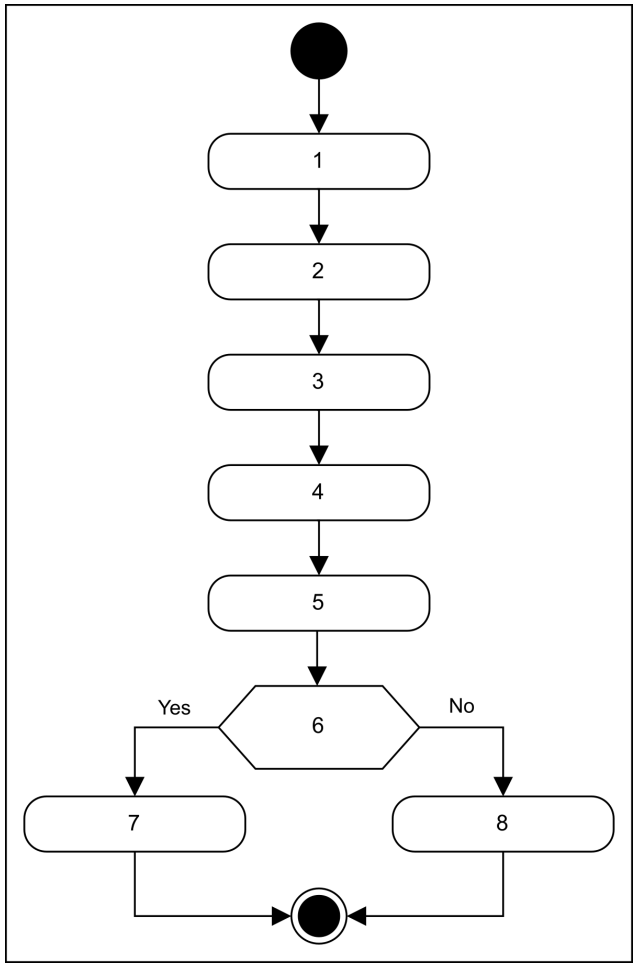


Figure 1.32: The activity diagram for an admin cancelling a show

Notice that the actions in the diagram above are numbered from 1 to 8. The slots shown below represent the activities, and the arrows represent the flow from one activity to the other.
Can you rearrange the given slots in the correct order they should appear in the activity diagram above?

Note: If you're stuck, click the "Show Solution" button.

Alternatively, you can also click the "Show complete diagram" button below to see the complete sequence diagram.

We've looked at some of the activity diagrams of our movie ticket booking system. In the next lesson, we will present the code for our designed classes in some of the most popular languages.

7 Code for the Movie Ticket Booking System

We've reviewed different aspects of the movie ticket booking system and observed the attributes attached to the problem using various UML diagrams. Let 's explore the more practical side of things, where we will work on implementing the movie ticket booking system using multiple languages. This is usually the last step in an object-oriented design interview process.
We have chosen the following languages to write the skeleton code of the different classes present in the movie ticket booking system:

- Java
- C#
- Python
- C++
- JavaScript

If you want to skip directly to the implementation and see the complete workflow, simply scroll down or click Executable Code: Movie Ticket Booking System.

7.1 Movie ticket booking system

This section will provide the skeleton code of the classes designed in the class diagram lesson.

> **Note:** For simplicity, we are not defining getter and setter functions. The reader can assume that all class attributes are private, accessed through their respective public getter methods, and modified only through their public method functions.

Enumerations

The following code defines the various enumerations used in the movie ticket booking system:

> **Note:** JavaScript does not support enumerations, so we will use the `Object.freeze()` method as an alternative that freezes an object and prevents further modifications.

Enum definitions

```
public enum PaymentStatus {
    PENDING, CONFIRMED, DECLINED, REFUNDED
}

public enum BookingStatus {
    PENDING, CONFIRMED, CANCELLED, DENIED, REFUNDED
}
```

```
public enum SeatStatus {
    AVAILABLE, BOOKED, RESERVED
}
```

Actors

This section contains the different people who will interact with our movie ticket systems, such as a `Customer`, `Admins`, and `TicketAgents`. All of these classes will inherit the properties of the `Person` class. The definition of these classes is given below:

Actors involved in the movie ticket booking system

```
// Person is an abstract class
public abstract class Person {
    public String name;
    public String address;
    public String phone;
    public String email;
}

public class Customer extends Person {
    public List<Booking> bookings = new ArrayList<>();

    public boolean createBooking(Booking booking) {
        return true;
    }
    public boolean updateBooking(Booking booking) {
        return true;
    }
    public boolean deleteBooking(Booking booking) {
        return true;
    }
}

public class Admin extends Person {
    public boolean addShow(ShowTime show) {
        return true;
    }
    public boolean updateShow(ShowTime show) {
        return true;
    }
    public boolean deleteShow(ShowTime show) {
        return true;
    }
    public boolean addMovie(Movie movie) {
        return true;
    }
    public boolean deleteMovie(Movie movie) {
        return true;
    }
}
```

```
public class TicketAgent extends Person {
    public boolean createBooking(Booking booking) {
        return true;
    }
}
```

Seat

The `Seat` will be an abstract class, which serves as a parent for three different types of seats: `Platinum`, `Gold`, and `Silver`. The definition of the `Seat` and its child classes is given below:

Seat and its derived classes

```
public abstract class Seat {
    public String seatNo;
    public SeatStatus status;

    public boolean isAvailable() {
        return status == SeatStatus.AVAILABLE;
    }

    public abstract void setSeat();
    public abstract void setRate();
}

public class Platinum extends Seat {
    public double rate;

    public void setSeat() { }
    public void setRate() { }
}


public class Gold extends Seat {
    public double rate;

    public void setSeat() { }
    public void setRate() { }
}

public class Silver extends Seat {
    public double rate;

    public void setSeat() { }
    public void setRate() { }
}
```

Movie, showtime, and movie ticket

Next, we will explore the `ShowTime`, `Movie`, and `MovieTicket` classes that provide the details of the movie to the customer. The definition of these classes is given below:

The Movie, ShowTime, and MovieTicket classes

```
public class Movie {
    public String title;
    public String genre;
    public Date releaseDate;
    public String language;
    public int duration;
    public List<ShowTime> shows = new ArrayList<>();

    public Movie(String title, String genre, Date releaseDate, String language, int
         duration, List<ShowTime> shows) {
        this.title = title;
        this.genre = genre;
        this.releaseDate = releaseDate;
        this.language = language;
        this.duration = duration;
        this.shows = shows;
    }
}

public class ShowTime {
    public int showId;
    public Date startTime;
    public Date date;
    public int duration;
    public List<Seat> seats = new ArrayList<>();

    public ShowTime(int showId, Date startTime, Date date, int duration, List<Seat>
         seats) {
        this.showId = showId;
        this.startTime = startTime;
        this.date = date;
        this.duration = duration;
        this.seats = seats;
    }

    public void showAvailableSeats() { }
}

public class MovieTicket {
    public int ticketId;
    public Seat seat;
    public Movie movie;
    public ShowTime show;

    public MovieTicket(int ticketId, Seat seat, Movie movie, ShowTime show) {
        this.ticketId = ticketId;
        this.seat = seat;
        this.movie = movie;
```

```
        this.show = show;
    }
}
```

City, cinema, and hall

This section contains classes like `Hall`, `Cinema`, and `City` that make up the infrastructure of our movie ticket system. The definition of these classes is given below:

The City, Cinema, and Hall classes

```
public class City {
    public String name;
    public String state;
    public int zipCode;
    public List<Cinema> cinemas = new ArrayList<>();

    public City(String name, String state, int zipCode, List<Cinema> cinemas) {
        this.name = name;
        this.state = state;
        this.zipCode = zipCode;
        this.cinemas = cinemas;
    }
}

public class Cinema {
    public int cinemaId;
    public List<Hall> halls = new ArrayList<>();
    public City city;

    public Cinema(int cinemaId, List<Hall> halls, City city) {
        this.cinemaId = cinemaId;
        this.halls = halls;
        this.city = city;
    }
}

public class Hall {
    public int hallId;
    public List<ShowTime> shows = new ArrayList<>();

    public Hall(int hallId, List<ShowTime> shows) {
        this.hallId = hallId;
        this.shows = shows;
    }

    public List<ShowTime> findCurrentShows() {
        return shows;
    }
}
```

Payment

The `Payment` class is another abstract class, with the `Cash` and `CreditCard` classes as its child. This takes the `PaymentStatus` enum to keep track of the payment status. The definition of this class is given below:

Payment class and its child classes

```
public abstract class Payment {
    public double amount;
    public Date timestamp;
    public PaymentStatus status;

    public abstract boolean makePayment();
}

public class Cash extends Payment {
    public boolean makePayment() {
        return true;
    }
}

public class CreditCard extends Payment {
    public String nameOnCard;
    public String cardNumber;
    public String billingAddress;
    public int code;

    public boolean makePayment() {
        return true;
    }
}
```

Notification

The `Notification` class is an abstract class that is responsible for sending notifications via email or phone/SMS after actions performed by either the admin or the customer. Its definition is given below:

Notification class and its child classes

```
public abstract class Notification {
    public int notificationId;
    public Date createdOn;
    public String content;

    public abstract void sendNotification(Person person);
}

public class EmailNotification extends Notification {
    public void sendNotification(Person person) { }
```

```
}

public class PhoneNotification extends Notification {
    public void sendNotification(Person person) { }
}
```

Booking

The `Booking` class is the main class of our movie ticket booking system and will display the information relating to a particular customer's booking. The definition of this class is given below:

The Booking class

```
import java.util.*;

public class Booking {
    public int bookingId;
    public int amount;
    public int totalSeats;
    public Date createdOn;
    public BookingStatus status;
    public Payment payment;
    public List<MovieTicket> tickets;
    public List<Seat> seats;

    public Booking(int bookingId, int amount, int totalSeats, Date createdOn,
        BookingStatus status, Payment payment, List<MovieTicket> tickets, List<Seat>
         seats) {
        this.bookingId = bookingId;
        this.amount = amount;
        this.totalSeats = totalSeats;
        this.createdOn = createdOn;
        this.status = status;
        this.payment = payment;
        this.tickets = tickets;
        this.seats = seats;
    }
}
```

Search and catalog

The `Catalog` class contains the movie information and implements the `Search` interface class to enable the search functionality based on the given criteria (title, language, genre, and release date). The definition of these two classes is given below:

The Search interface and Catalog class

```java
public interface Search {
    List<Movie> searchMovieTitle(String title);
    List<Movie> searchMovieLanguage(String language);
    List<Movie> searchMovieGenre(String genre);
    List<Movie> searchMovieReleaseDate(Date date);
}


public class Catalog implements Search {
    public HashMap<String, List<Movie>> movieTitles = new HashMap<>();
    public HashMap<String, List<Movie>> movieLanguages = new HashMap<>();
    public HashMap<String, List<Movie>> movieGenres = new HashMap<>();
    public HashMap<Date, List<Movie>> movieReleaseDates = new HashMap<>();

    public List<Movie> searchMovieTitle(String title) { }
    public List<Movie> searchMovieLanguage(String language) { }
    public List<Movie> searchMovieGenre(String genre) { }
    public List<Movie> searchMovieReleaseDate(Date date) { }
}
```

7.2 Executable code: Movie ticket booking system

Below is a fully self-contained, runnable program in Java, C#, C++, Python, and JavaScript that demonstrates the core workflows of the Movie Ticket Booking system. The main driver code of the system resides in the `Driver.java`, `Driver.cs`, `Driver.py`, `Driver.cpp`, and `Driver.js` for all respective languages. You can click the "Run" button to execute the code.

What does this code show

- **System initialization:** Sets up movies, a cinema with halls and seats, a customer, and an admin.
- **Scenario 1 (Customer searches and books a ticket):** The customer searches for a movie, selects a showtime and seats, creates a booking, and the system confirms the booking.
- **Scenario 2 (Payment for a booking):** The customer pays for the booking; the payment is processed, and the system sends a confirmation notification.
- **Scenario 3 (Customer cancels a booking):** The customer cancels a booking, the system updates statuses, handles a refund if eligible, and sends a cancellation notification.

Hint: Try customizing the code!
This code is designed for experimentation, not just reading. You can edit, extend, or modify any part of the example.

Executable code: Movie Ticket Booking System

```java
import java.util.*;
import java.text.SimpleDateFormat;

public class Driver {
    public static void main(String[] args) {
        SimpleDateFormat sdf = new SimpleDateFormat("yyyy-MM-dd");

        // ----- SETUP PHASE -----
        System.out.println("=========================================================");
        System.out.println("MOVIE TICKET BOOKING SYSTEM - SCENARIO DEMONSTRATION");
        System.out.println("=========================================================");

        // 1. Create city, cinema, hall, seats
        City city = new City("Seattle", "WA", 98101, new ArrayList<>());
        Cinema cinema = new Cinema(1, new ArrayList<>(), city);
        city.cinemas.add(cinema);

        Hall hall = new Hall(101, new ArrayList<>());
        cinema.halls.add(hall);

        // 2. Create movies
        Movie inception = new Movie(
            "Inception", "Sci-Fi",
            parseDate("2010-07-16"), "English", 148, new ArrayList<>()
        );
        Movie interstellar = new Movie(
            "Interstellar", "Sci-Fi",
            parseDate("2014-11-07"), "English", 169, new ArrayList<>()
        );

        // 3. Create ShowTimes (each with seats)
        List<Seat> seatsShow1 = Arrays.asList(new Platinum(), new Gold(), new Silver
            ());
        for (int i = 0; i < seatsShow1.size(); i++) {
            seatsShow1.get(i).seatNo = "A" + (i+1);
            seatsShow1.get(i).status = SeatStatus.AVAILABLE;
            seatsShow1.get(i).setRate();
        }
        ShowTime show1 = new ShowTime(1001, new Date(), new Date(), 150, seatsShow1)
            ;
        hall.shows.add(show1);
        inception.shows.add(show1);

        // 4. Create Catalog and add movies
        Catalog catalog = new Catalog();
        catalog.movieTitles = new HashMap<>();
        catalog.movieTitles.put("Inception", Arrays.asList(inception));
        catalog.movieTitles.put("Interstellar", Arrays.asList(interstellar));

        // 5. Create admin, customer, ticket agent
        Admin admin = new Admin();
        admin.name = "Bob (Admin)";
        admin.email = "bob.admin@cinema.com";

        Customer customer = new Customer();
```

```java
customer.name = "Alice (Customer)";
customer.email = "alice@example.com";
customer.bookings = new ArrayList<>();

TicketAgent agent = new TicketAgent();
agent.name = "Eve (Ticket Agent)";
agent.email = "eve.agent@cinema.com";

// ----- SCENARIO 1: CUSTOMER SEARCHES FOR A MOVIE -----
System.out.println("----- SCENARIO 1: Customer Searches For A Movie -----");
System.out.printf("%s is searching for the movie 'Inception':", customer.
    name);
List<Movie> found = catalog.searchMovieTitle("Inception");
for (Movie m : found) {
    System.out.printf(" - Found: %s (%s, Released: %s)", m.title, m.genre,
        sdf.format(m.releaseDate));
}

// ----- SCENARIO 2: CUSTOMER BOOKS A TICKET -----
System.out.println("----- SCENARIO 2: Customer Books A Ticket -----");
Seat chosenSeat = show1.seats.get(0); // Platinum
if (chosenSeat.isAvailable()) {
    chosenSeat.status = SeatStatus.BOOKED;

    // Create ticket and booking
    MovieTicket ticket = new MovieTicket(2001, chosenSeat, inception, show1)
        ;
    List<MovieTicket> tickets = new ArrayList<>();
    tickets.add(ticket);

    // Payment
    CreditCard payment = new CreditCard();
    payment.amount = 15.0;
    payment.nameOnCard = customer.name;
    payment.cardNumber = "1234-5678-9012-3456";
    payment.billingAddress = "123 1st Ave, Seattle, WA";
    payment.code = 123;
    payment.status = PaymentStatus.PENDING;
    payment.timestamp = new Date();
    payment.makePayment();
    payment.status = PaymentStatus.CONFIRMED;

    // Booking
    Booking booking = new Booking(3001, 15, 1, new Date(), BookingStatus.
        CONFIRMED,
            payment, tickets, Arrays.asList(chosenSeat));
    customer.bookings.add(booking);

    // Notification
    EmailNotification notification = new EmailNotification();
    notification.content = String.format("Booking confirmed for %s!Seat: %s
        (%s)Show ID: %d",
        inception.title, chosenSeat.seatNo, chosenSeat.getClass().
            getSimpleName(), show1.showId);
    notification.sendNotification(customer);
```

```
        System.out.printf("%s successfully booked a ticket for: %s", customer.
            name, inception.title);
        System.out.printf(" - Seat: %s (%s), Show ID: %d", chosenSeat.seatNo,
            chosenSeat.getClass().getSimpleName(), show1.showId);
        System.out.printf(" - Payment: $%.2f by Credit Card ending %s", payment.
            amount, payment.cardNumber.substring(payment.cardNumber.length()-4));
    } else {
        System.out.println("Seat is not available!");
    }

    // ----- SCENARIO 3: ADMIN ADDS A NEW SHOW -----
    System.out.println("----- SCENARIO 3: Admin Adds A New Show -----");
    ShowTime newShow = new ShowTime(1002, new Date(), new Date(), 148, new
        ArrayList<>());
    boolean showAdded = admin.addShow(newShow);
    if (showAdded) {
        hall.shows.add(newShow);
        inception.shows.add(newShow);
        System.out.printf("%s added a new show for: %s (Show ID: %d)", admin.
            name, inception.title, newShow.showId);
    }

    // ----- SCENARIO 4: TICKET AGENT CREATES A BOOKING FOR WALK-IN -----
    System.out.println("----- SCENARIO 4: Ticket Agent Creates A Walk-In Booking
         -----");
    ShowTime showInterstellar = new ShowTime(1003, new Date(), new Date(), 169,
        new ArrayList<>());
    hall.shows.add(showInterstellar);
    interstellar.shows.add(showInterstellar);

    List<Seat> seatsShow2 = Arrays.asList(new Gold(), new Silver());
    for (int i = 0; i < seatsShow2.size(); i++) {
        seatsShow2.get(i).seatNo = "B" + (i+1);
        seatsShow2.get(i).status = SeatStatus.AVAILABLE;
        seatsShow2.get(i).setRate();
    }
    showInterstellar.seats = seatsShow2;

    Seat walkInSeat = showInterstellar.seats.get(1); // Silver
    if (walkInSeat.isAvailable()) {
        walkInSeat.status = SeatStatus.BOOKED;

        MovieTicket ticket = new MovieTicket(2002, walkInSeat, interstellar,
            showInterstellar);
        List<MovieTicket> tickets = new ArrayList<>();
        tickets.add(ticket);

        Cash cashPayment = new Cash();
        cashPayment.amount = 10.0;
        cashPayment.status = PaymentStatus.PENDING;
        cashPayment.timestamp = new Date();
        cashPayment.makePayment();
        cashPayment.status = PaymentStatus.CONFIRMED;

        Booking agentBooking = new Booking(3002, 10, 1, new Date(),
            BookingStatus.CONFIRMED,
```

```
                cashPayment, tickets, Arrays.asList(walkInSeat));
            boolean agentBooked = agent.createBooking(agentBooking);
            if (agentBooked) {
                System.out.printf("%s booked ticket for: %s (walk-in)", agent.name,
                    interstellar.title);
                System.out.printf(" - Seat: %s (%s), Show ID: %d", walkInSeat.seatNo,
                     walkInSeat.getClass().getSimpleName(), showInterstellar.showId);
                System.out.printf(" - Payment: $%.2f by Cash", cashPayment.amount);
            }
        }

        // ----- SCENARIO 5: ADMIN DELETES A MOVIE -----
        System.out.println("----- SCENARIO 5: Admin Deletes A Movie -----");
        boolean movieDeleted = admin.deleteMovie(interstellar);
        if (movieDeleted) {
            catalog.movieTitles.remove("Interstellar");
            System.out.printf("%s deleted movie: %s", admin.name, interstellar.title
                );
        }

        System.out.println("=====================================================");
        System.out.println(" END OF MOVIE TICKET BOOKING DEMO RUN");
        System.out.println("=====================================================");
    }

    // Helper to parse date without deprecated API
    public static Date parseDate(String dateStr) {
        try {
            return new SimpleDateFormat("yyyy-MM-dd").parse(dateStr);
        } catch (Exception ex) {
            return new Date();
        }
    }
}
```

7.3 Wrapping up

In this chapter, we've explored the complete design of a movie ticket booking system. Also , we've looked at how a basic system can be visualized using various UML diagrams and designed using object-oriented principles and design patterns.

8 Summary: Designing a Movie Ticket Booking System

The movie ticket booking system is designed to facilitate searching for movies, viewing showtimes, and booking tickets across multiple cinemas and their halls. The system allows online

customers and in-person ticket agents to manage bookings and payments, enabling administrators to control movie and show listings. It incorporates real-time seat selection, various payment methods, and an automated notification system for relevant events.

8.1 Key requirements

The following are the key requirements of the case study:

1. The system must support multiple cinemas in different cities, each with multiple halls, where each movie can have various shows, but a hall hosts only one show at a time.
2. The system must display all available showtimes for selected movies across all cinemas.
3. Users can search for movies by title, language, genre, or release date.
4. Users can book multiple available seats for any showtime in a single transaction, online or in person via a ticket agent.
5. Seats are categorized into "Silver," "Gold," and "Platinum" with fixed costs. Only one ticket can be allocated per seat, and the system prevents duplicate bookings for the same seat and show.
6. Online customers can pay by credit card only, while in-person customers can pay by cash or credit card through a ticket agent.
7. The system must visually distinguish between available and booked seats.
8. Admin users must be able to add, delete, or update shows and movies.
9. The system must generate notifications for new movie releases, booking confirmations, cancellations, and booking modifications.

8.2 System actors

The following are the main actors in the case study:

- **Customer:** Searches for movies, creates bookings, reserves seats, pays for tickets, and views/modifies/cancels bookings.
- **Ticket agent:** This person assists customers by creating bookings, searching for movies, reserving seats on their behalf, and accepting payments, but they cannot modify or cancel a booking.
- **Admin:** Manages movies and showtimes by adding, deleting, or updating them.

8.3 Important classes and relationships

The following table describes the main classes and their relationships with other classes, including inheritance, association, aggregation, and composition.

Class	Description	Important Relationships
`Seat`	Abstract class for a seat with types: `Silver`, `Gold`, `Platinum`.	Inherited by `Silver`, `Gold`, and `Platinum`
`ShowTime`	Represents a scheduled movie show with time, date, and seat status map.	Associated with `Movie`, `Seat`, and `MovieTicket`
`Hall`	A cinema hall with seats and current shows.	Composed within `Cinema`, composed of `ShowTime`
`Cinema`	Represents a cinema with multiple halls.	Composed of `Hall`, associated with `City`
`City`	Represents a geographic location.	Has a one-way association with `Cinema`
`Movie`	Stores movie metadata and running shows.	Associated with `ShowTime`, `MovieTicket`, and aggregated in `Catalog`
`MovieTicket`	Represents a booked seat with ticket ID, show, and seat info.	Linked to `Booking`, `ShowTime`, and `Movie`
`Payment`	Abstract class for payment methods.	Subclassed by Cash and `CreditCard`, associated with `Booking`
`Booking`	Core class to manage seat selection, payment, and booking status.	Associated with `MovieTicket`, `ShowTime`, `Payment`
`Person`	Base class for system users.	Inherited by `Customer`, `Admin`, and `TicketAgent`
`Notification`	Abstract class for notifications.	Subclassed by `EmailNotification`, `PhoneNotification`
`Catalog`	Implements search functionality.	Aggregates `Movie`, implements `Search` interface
`Search`	Interface for search operations.	Implemented by `Catalog`

8.4 Design highlights

This section outlines the key design methodologies and principles in designing a movie ticket booking System.

Design approach

The system follows a bottom-up design methodology, from defining entities like seats, halls, and users to integrating them into a cohesive system structure.

Design principles

The following are the design principles that are applied to the design of the case study:

- **Single Responsibility principle (SRP):** Each class handles a distinct responsibility, e.g., `Seat` handles seat info, `Booking` manages booking logic.
- **Open/Closed principle (OCP):** The `Seat` class allows for extension (Silver, Gold, Platinum) without modification.
- **Liskov Substitution principle (LSP):** Subclasses like `CreditCard` and `Cash` can substitute `Payment` without affecting the system.
- **Interface Segregation principle (ISP):** The `Search` interface defines focused methods for searching movies without coupling to unrelated functionalities.
- **Dependency Inversion principle (DIP):** High-level classes like `Catalog` depend on abstractions (`Search` interface), not concrete implementations.

Design patterns

The following are the design patterns that are applied to the design of the case study:

- **Factory pattern:** This pattern creates various `Seat` types based on category (Silver, Gold, Platinum).
- **Observer pattern:** The notifications system uses observers to send updates on movie releases, bookings, and changes.
- **Strategy pattern:** The `Search` interface allows interchangeable search strategies (by title, genre, language, etc.).
- **Singleton pattern:** This pattern ensures that central classes like `Catalog` or the notification dispatcher have only one global instance managing state.
- **Builder pattern:** Facilitates the step-by-step construction of complex `Booking` objects, especially involving multiple seat selections.

8.5 Object interactions

The following are the main object interactions:

- **Ticket issuance:** The customer or ticket agent selects a movie and showtime, reserves seat(s), confirms booking, and receives the ticket.

- **Fee calculation:** The system computes the total cost and handles payment based on seat category (Silver/Gold/Platinum).
- **Notification generation:** Upon booking, cancellation, or movie release, the system dispatches relevant notifications to users.
- **Booking modification:** When the customer modifies the booking, the system updates the seat map and notifies the user.
- **Cancellation:** The booking is canceled, the seat status is reverted, a refund (if applicable) is initiated, and a cancellation notice is sent.

8.6 System workflow

The following are the main workflows that we have worked on in the case study:

- **Customer booking flow:**
  - Search for a movie using filters (title, genre, language, date).
  - Select a showtime from available listings.
  - Choose seat(s) on the seating map.
  - Make a payment via credit card.
  - Receive booking confirmation and ticket.
- **Admin management flow:**
  - Add or update a movie and its shows.
  - Modify or delete existing shows (which may trigger notifications).
  - Remove a movie entirely, canceling all associated shows.
- **Ticket agent flow:**
  - Perform movie searches and create bookings for walk-in customers.
  - Accept payment in cash or by credit card.
  - View and manage customer bookings (no modification/cancellation rights).
- **System notification flow:**
  - Automatically send booking confirmation, modification, or cancellation alerts.
  - Notify opted-in users about new movie releases.

9 Key Insights and Tips: Designing a Movie Ticket Booking System

You have successfully reviewed the case study for designing a movie ticket booking system. This lesson provides a comprehensive summary of the critical aspects involved in designing a movie ticket booking system. It will show you common mistakes to avoid, important tips for interviews, a short quiz to test what you know, and other case studies that can help you understand the ideas better.

9.1 Common mistakes

Avoiding these common mistakes will help you develop a more efficient and scalable movie ticket booking system:

- **Ignoring concurrency for seat booking:** Failing to prevent multiple users from booking the same seat simultaneously. Optimistic locking or versioning should be implemented to manage concurrent seat allocations, ensuring that only one customer can successfully book a seat.
- **Inadequate handling of payment methods:** Limiting payment options or not abstracting the payment process. You should design an abstract `Payment` class with concrete child classes like `CreditCard` and `Cash` to support various payment methods.
- **Missing notification mechanisms:** Not incorporating a way to inform users about important events. You should design a `Notification` abstract class with concrete implementations like `EmailNotification` and `PhoneNotification` to send updates for new movies, bookings, or cancellations.
- **Not considering booking and payment statuses:** Bookings and payments have no clear states, leading to ambiguous transaction tracking. To track the state of transactions, you should use enumerations like `BookingStatus` (pending, confirmed, canceled, declined, refunded) and `PaymentStatus` (confirmed, declined, pending, refunded).
- **Not implementing a flexible search mechanism:** Hardcoding search criteria instead of allowing for varied search options. You should create a `Search` interface implemented by a `Catalog` class that allows searching for movies by title, language, genre, or release date.

9.2 Key interview tips

The following table shows some tips for preparing for an interview where you're asked to design a movie ticket booking system.

9.3 Test your understanding

Take this quick quiz to check your understanding of designing a movie ticket booking system:

Tip	Explanation
Justify your class design choices.	When presenting your class diagram, explain why you chose certain classes (e.g., Abstract `Seat` with derived types, Payment hierarchy) and their relationships. This shows thoughtful design and understanding of object-oriented principles.
Detail the flow of a booking.	Be prepared to walk through the entire sequence of a customer creating and paying for a booking, explaining the interactions between different objects. This demonstrates a clear understanding of the system's operational flow.
Consider extensibility with design patterns.	Mention how design patterns, like the Strategy pattern for seat pricing, can be applied to make the system more flexible and maintainable. This shows foresight and an understanding of scalable architecture.
Address edge cases.	Be ready to discuss how the system handles scenarios like declined payments, including reverting seat statuses. This showcases attention to detail and robust error handling.
Think about different user interfaces.	Explain how the system accommodates online customer and walk-in bookings via a ticket agent, considering their different payment methods and permissions. This shows a comprehensive view of system access.

Table 1.1: Tips for Designing and Explaining a Booking System

Designing a Movie Ticket Booking System

Question 1: What design principle is suggested for calculating varying ticket fares for different seat types (silver, gold, platinum)?

- Singleton pattern
- Observer pattern
- **[CORRECT]** Strategy pattern
  Explanation: The Strategy design pattern is recommended to design separate algorithms for calculating the price of each seat type, allowing for flexibility based on factors like popularity or taxes.
- Factory pattern

Question 2: What is the relationship between the Create booking use case and the Reserve

seat use case?

- Extends
- Generalization
- **[CORRECT]** Include
  Explanation: When a user performs the "Create booking" action, it inherently involves the "Reserve seat" action, because a complete and valid booking cannot be created without first reserving the desired seats.
- Association

Question 3: How does the system prevent two customers from reserving the same seat concurrently?

- By using a simple FIFO queue for seat requests.
- By assigning a unique transaction ID to each booking.
- **[CORRECT]** By implementing optimistic locks or versioning.
  Explanation: This allows multiple customers to view the same seat selection map (each assigned a version number) but ensures that only the first customer to successfully complete their payment process updates the system's version. If another customer attempts to book the same seat with an outdated version, their transaction will be rejected, requiring them to restart their selection process.
- By manually checking seat availability at the time of payment.

Related case studies

Explore these related case studies to deepen your understanding of the design principles applied in the movie ticket booking system:

- **Designing a flight reservation system:** This involves managing available flights, seats, and bookings, which is highly analogous to a movie ticket booking system.
- **Designing a ride sharing service (e.g., Uber/Lyft):** While different in domain, it involves managing real-time availability of resources (cars/drivers) and matching them with user requests, which shares principles with dynamic seat availability and booking.
- **Designing the Amazon locker service:** This case study deals with allocating limited resources (lockers) to users, similar to allocating seats in a movie theatre.
- **Designing a hotel management system:** This is very similar to a movie booking system as it deals with room inventory, availability, reservation, and payment processing.

- **Designing a restaurant management system:** This includes managing table reservations, staff scheduling, and inventory, all of which involve resource allocation and booking.